Feature 1:

Prompt weak:

i want to add this feature Due dates + overdue filter, write me several user stories

Response to weak prompt:

Here are user stories for the due dates + overdue filter feature, grouped by capability:

Setting due dates

1. As a user, I want to add a due date to a task when creating it, so that I can plan my work around deadlines.
2. As a user, I want to add, change, or remove a due date on an existing task, so that I can adjust when priorities shift.
3. As a user, I want to pick a due date from a calendar picker (with quick options like "Today," "Tomorrow," "Next week"), so that setting deadlines is fast and error-free.

Seeing due dates

4. As a user, I want to see the due date displayed on each task in my list, so that I can tell at a glance what's coming up.
5. As a user, I want tasks due today to be visually highlighted, so that I know what needs my attention right now.
6. As a user, I want overdue tasks to be clearly marked (e.g., red date label), so that missed deadlines stand out from upcoming ones.

Overdue filter

7. As a user, I want to filter my task list to show only overdue tasks, so that I can focus on catching up on what I've missed.
8. As a user, I want to combine the overdue filter with my other filters (e.g., by project or tag), so that I can narrow down overdue work in a specific area.
9. As a user, I want to see a count of overdue tasks on the filter (e.g., "Overdue (4)"), so that I know the size of my backlog without opening it.

Sorting & behavior

10. As a user, I want to sort my tasks by due date, so that the most urgent items appear first.
11. As a user, I want tasks without a due date to be handled predictably when sorting (e.g., grouped at the bottom), so that they don't clutter my deadline view.
12. As a user, I want a completed task to no longer count as overdue, so that my overdue list only reflects work that's actually pending.

Prompt strong:

I want to add the feature Due dates + overdue filter,

with Expected backend work:

-Add optional due_date validation. Support create/update. -Decide whether overdue is computed in the backend or UI. -Optional query filter for overdue.

with expected frontend work:

-Add due date to the modal. -Show due date or overdue pill on cards. -Add an overdue filter or visual indicator.

and create test for this feature:

-Valid due date, invalid date format, overdue detection, update due date, filter returns only overdue tasks.

i want from you to give me several user stories in the same format and quality as this example.

Example: Story: As a team member, I want to create a task so that I can track work. Acceptance Criteria: - Title is required; missing or blank title returns HTTP 422. - Description is optional. - A created task appears in the task list with status, priority, and assignee.

Now generate five more stories in the same format.

Constraints:

Use "team member" as the user role.

Do not mention login, authentication, user accounts, admin roles, notifications, mobile, or real-time updates.

Include at least one failure case across the generated stories.

Output format: Return each story with Story and Acceptance Criteria headings.

Response to strong prompt:

Here are five user stories for the Due dates + overdue filter feature, in the same format as your example.

Story 1: As a team member, I want to set an optional due date when creating a task so that I can plan when work should be finished.

Acceptance Criteria:

- due_date is optional; a task created without one is accepted and stored with due_date as null.
- A valid ISO 8601 date (e.g., 2026-07-30) is accepted and returned in the task response.
- An invalid date format (e.g., "next week", 30-07-2026) returns HTTP 422 with a clear validation message.

Story 2: As a team member, I want to add, change, or remove the due date on an existing task so that I can adjust plans when priorities shift.

Acceptance Criteria:

- PATCH /tasks/{id} with a valid due_date updates the task and returns the new value.
- PATCH with due_date set to null clears the due date.
- A PATCH that does not include due_date leaves the existing due date unchanged.
- Updating the due date of a missing task id returns HTTP 404.

Story 3: As a team member, I want overdue tasks to be clearly identified so that I can see which work has slipped.

Acceptance Criteria:

- A task is overdue when its due_date is before today and its status is not Done.
- Tasks with status Done are never marked overdue, regardless of due date.
- Tasks with no due_date are never marked overdue.

Story 4: As a team member, I want to filter the task list to only overdue tasks so that I can focus on late work first.

Acceptance Criteria:

- GET /tasks?overdue=true returns only tasks that meet the overdue definition.
- If no tasks are overdue, the response is HTTP 200 with an empty list, not 404.
- An invalid overdue value (e.g., ?overdue=banana) returns HTTP 422.

Story 5: As a team member, I want to see the due date on each task card so that I can judge urgency at a glance without opening the task.

Acceptance Criteria:

- Cards for tasks with a due date show the date in a readable format; cards without one show no date element.
- Overdue tasks display a visually distinct overdue pill on the card.
- The due date field appears in the create/edit modal, is optional, and an invalid entry surfaces the server's 422 message without closing the modal.

Strong prompt for User stories table:

You are a senior developer reviewing requirements before implementation.

Context: The requirements should support a simple FastAPI backend and simple web frontend. The app has no authentication, no user accounts, no multi-tenancy, no real-time updates, no mobile app, and no notifications.

Task: Review the user stories below for testability and scope alignment. Check each story for:

- Is the story in scope?
- Are the acceptance criteria specific and testable?
- Is there at least one meaningful edge case or failure case across the set?
- Are any words too vague, such as "works", "properly", or "as expected"?
- Did the AI assume a feature that was not requested?

Constraints:

- Do not generate a new story set.
- Suggest minimal edits only.

Output format:

Return a table with columns: Story ID, Pass / Needs revision, Issue, Minimal edit. User stories: Here are five user stories for the Due dates + overdue filter feature, in the same format as your example.

Story 1: As a team member, I want to set an optional due date when creating a task so that I can plan when work should be finished. Acceptance Criteria:

- due_date is optional; a task created without one is accepted and stored with due_date as null.
- A valid ISO 8601 date (e.g., 2026-07-30) is accepted and returned in the task response.
- An invalid date format (e.g., "next week", 30-07-2026) returns HTTP 422 with a clear validation message.

Story 2: As a team member, I want to add, change, or remove the due date on an existing task so that I can adjust plans when priorities shift. Acceptance Criteria:

- PATCH /tasks/{id} with a valid due_date updates the task and returns the new value.
- PATCH with due_date set to null clears the due date.
- A PATCH that does not include due_date leaves the existing due date unchanged.
- Updating the due date of a missing task id returns HTTP 404.

Story 3: As a team member, I want overdue tasks to be clearly identified so that I can see which work has slipped. Acceptance Criteria:

- A task is overdue when its due_date is before today and its status is not Done.
- Tasks with status Done are never marked overdue, regardless of due date.
- Tasks with no due_date are never marked overdue.

Story 4: As a team member, I want to filter the task list to only overdue tasks so that I can focus on late work first. Acceptance Criteria:

- GET /tasks?overdue=true returns only tasks that meet the overdue definition.
- If no tasks are overdue, the response is HTTP 200 with an empty list, not 404.
- An invalid overdue value (e.g., ?overdue=banana) returns HTTP 422.

Story 5: As a team member, I want to see the due date on each task card so that I can judge urgency at a glance without opening the task. Acceptance Criteria:

- Cards for tasks with a due date show the date in a readable format; cards without one show no date element.
- Overdue tasks display a visually distinct overdue pill on the card.
- The due date field appears in the create/edit modal, is optional, and an invalid entry surfaces the server's 422 message without closing the modal.

Response table:

Story ID	Pass / Needs revision	Issue	Minimal edit
Story 1	Needs revision (minor)	"A clear validation message" is not testable — what makes it clear?	Change to: "returns HTTP 422 with a detail message that names the due_date field."
Story 2	Pass	In scope, each criterion maps directly to a PATCH test, includes the 404 failure case. Note: "remove via null" requires the update model to distinguish *field absent* from *field set to null* — worth confirming your TaskUpdate/exclude_unset handling supports this before promising it.	No wording change needed; add a note/assumption about the null-vs-absent distinction.

Story 3	Needs revision (minor)	"Before today" is ambiguous — today in which timezone? Server date vs. client date can disagree, which makes the overdue test flaky near midnight. Also, the criteria imply backend computation but never state it, and that was an unrequested AI assumption (the brief says "decide whether overdue is computed in the backend or UI").	Add: "Overdue is computed by the backend using the server's current UTC date." (Or state your chosen alternative — record the decision in the mini-ADR.)
Story 4	Pass	In scope and testable; empty-list 200 and invalid-value 422 are both concrete. The 422 for `?overdue=banana` comes free from FastAPI's bool query parsing — fine, but know you're testing framework behavior, not custom code.	None required.
Story 5	Needs revision	Two vague phrases: "readable format" and "visually distinct" are not testable as written. Also, the third criterion (modal behavior) belongs to a different flow than the card display — it bundles two concerns into one story.	Specify the format: "shows the date as YYYY-MM-DD" (or your chosen fixed format). Replace "visually distinct overdue pill" with "a pill element containing the text 'Overdue'". Optionally split the modal criterion into its own story or fold it into Story 1/2.

Cross-set checks:

- **Scope:** All five stories stay inside the no-auth, no-notifications, single-list scope. No violations found.
- **Failure cases:** Yes — invalid date format (S1), missing task 404 (S2), invalid query value (S4). Requirement met.
- **Unrequested assumptions:** One meaningful one — backend-computed overdue (S3), which the project brief explicitly leaves as a decision, not a given. That's a good candidate for the "AI assumption I corrected/confirmed" entry required in docs/user-stories.md. The date-only (no time) assumption is also implicit in S1's examples; state it once explicitly.
- **Manual-check criteria:** S5's criteria are frontend and will be verified by browser checks, not pytest — that's acceptable, but mark them as manual checks in docs/verification.md rather than expecting automated coverage.

Apply the four minimal edits above and the set is ready to save. This review exchange itself is good material for docs/prompt-log.md, by the way — it shows a strong prompt (role, checklist, constraints, output format) and what you did with the output.

Still need your second feature pick when you're ready.

Editing the user stories

edit story 1, the validation message should include the due_date field to know that the error belong to this feature

edit story 3, to be more precise some tasks may be in Done but after their due date so they should be flagged as "completed late"

Result to editing the user stories

Story 1: As a team member, I want to set an optional due date when creating a task so that I can plan when work should be finished.

Acceptance Criteria:

- due_date is optional; a task created without one is accepted and stored with due_date as null.
- A valid ISO 8601 date (e.g., 2026-07-30) is accepted and returned in the task response.
- An invalid date format (e.g., "next week", 30-07-2026) returns HTTP 422 with a detail message that names the due_date field, so the error is clearly attributable to this feature.

Story 2: As a team member, I want to add, change, or remove the due date on an existing task so that I can adjust plans when priorities shift.

Acceptance Criteria:

- PATCH /tasks/{id} with a valid due_date updates the task and returns the new value.
- PATCH with due_date set to null clears the due date.
- A PATCH that does not include due_date leaves the existing due date unchanged.
- Updating the due date of a missing task id returns HTTP 404.

Story 3: As a team member, I want tasks to be flagged relative to their due date so that I can see which work has slipped and which work finished late.

Acceptance Criteria:

- A task is overdue when its due_date is before the server's current UTC date and its status is not Done.
- A task with status Done whose due_date is before the server's current UTC date is flagged as "completed late", not overdue.

- A task with status Done on or before its due_date carries neither flag.
- Tasks with no due_date carry neither flag.

Story 4: As a team member, I want to filter the task list to only overdue tasks so that I can focus on late work first.

Acceptance Criteria:

- GET /tasks?overdue=true returns only tasks whose due_state is overdue; completed-late tasks are excluded.
- If no tasks are overdue, the response is HTTP 200 with an empty list, not 404.
- An invalid overdue value (e.g., ?overdue=banana) returns HTTP 422.

Story 5: As a team member, I want to see the due date and lateness state on each task card so that I can judge urgency at a glance without opening the task.

Acceptance Criteria:

- Cards for tasks with a due date show the date as YYYY-MM-DD; cards without one show no date element.
- Tasks whose due_state is overdue display a pill element containing the text "Overdue".
- Done tasks whose due_state is completed_late display a pill element containing the text "Completed late".
- The due date field appears in the create/edit modal, is optional, and an invalid entry surfaces the server's 422 detail message without closing the modal.

ADR

Reject option it has sqlite for persistence data and we do not want that out of scope, so accept option A,

ADR - Option A written by me

We are adding a new feature to our task management API that will enable due dates and due states (e.g., tracking if a work is overdue or completed late). We must choose how to handle and store this new data. A basic in-memory storage method is used in our current application. Due to an impending deadline, we have to decide between moving to a persistent database (Option B) or continuing with this in-memory method (Option A).

We proceed with Option A.

We are adding a new feature to our task management API that will enable due dates and due states (e.g., tracking if a work is overdue or completed late). We must choose how to handle and store this new data. A basic in-memory storage method is used in our current application. Due to an impending deadline, we have to decide between moving to a persistent database

(Option B) or continuing with this in-memory method (Option A).

We decided to keep the in-memory dictionary and extend it with due_date as a stored field and due_state as a value computed at response time. We chose this because it is the simplest option, it keeps our existing pytest suite and reset fixture working without changes, the app still runs locally with a single uvicorn command, and the whole team is already familiar with the current storage code from Modules 2 and 3.

During planning we rejected or corrected two AI suggestions. First, the AI proposed a SQLite-based storage layer (Option B); we rejected it because rewriting the storage foundation a week before the deadline added risk without improving what the project actually assesses. Second, the AI originally assumed that Done tasks are never flagged regardless of their due date; we corrected this so that Done tasks past their due date are flagged as "completed late" instead of overdue.

Agent prompts to code feature 1:

I am working in my Task Tracker repository (branch: mid-course-project) for an AI-Assisted Coding course.

Current project:

- Backend: Python/FastAPI, Pydantic v2, in-memory storage in app/storage.py (_tasks dict).

- Files: app/main.py (5 CRUD routes), app/models.py, app/storage.py, app/business_rules.py, tests/.

- Status values exactly: ToDo, InProgress, Done. Priority: Low, Medium, High.

- Frontend: frontend/index.html, vanilla HTML/CSS/JS Kanban board with drag-and-drop and create/edit modal.

- Baseline: full pytest suite passing before this feature.

Feature I am adding (decided, do not propose alternatives):

- Optional due_date field: calendar date (ISO YYYY-MM-DD), no time component, nullable.

- due_state computed at response time in the backend, never stored:

  - "overdue" = due_date before server's current UTC date AND status is not Done

  - "completed_late" = due_date before server's current UTC date AND status is Done

  - null = no due_date, or due_date is today or later

- GET /tasks gets an optional overdue=true query filter; returns only "overdue" tasks (completed_late excluded); empty result is 200 with [].

- Storage stays in-memory. DO NOT add a database, ORM, or new dependencies.

Workflow rules:

- Work in small steps, one file or one function per prompt.

- Do not rewrite whole files. Do not touch unrelated routes or tests.

- Treat your output as a draft; I will inspect, run, and test before accepting.

Prompt to write tests:

You are a senior Python developer adding pytest tests to an existing FastAPI test suite.

Context files:
@app/main.py
@app/models.py
@app/storage.py
@tests/conftest.py
@tests/test_tasks.py

The suite already has passing tests and fixtures (_reset_storage autouse, client, created_task, plus one existing overdue-filter test). Do NOT regenerate conftest.py. Do NOT modify or rename any existing test.

Task: ADD these new tests for the due_date feature to tests/test_tasks.py:

Due date on create:
- test_create_task_with_valid_due_date_returns_201_and_echoes_date
  (POST with "due_date": "2026-07-30"; assert 201, body["due_date"] == "2026-07-30",
   body["due_state"] is None since the date is in the future)
- test_create_task_without_due_date_returns_201_with_null_due_date
  (assert due_date is None and due_state is None)
- test_create_task_invalid_due_date_format_returns_422_naming_due_date
  (POST with "due_date": "banana"; assert 422 and that "due_date" appears in the
   error detail loc)

Due state computation:

- test_past_due_date_not_done_has_due_state_overdue
  (create task with a past date such as "2020-01-01"; assert due_state == "overdue")
- test_past_due_date_done_has_due_state_completed_late
  (create with past date, PATCH status through ToDo -> InProgress -> Done using the
   valid transition chain; assert final due_state == "completed_late")

Due date update:
- test_patch_due_date_updates_value
  (create without date, PATCH {"due_date": "2026-08-01"}, assert 200 and new value)
- test_patch_due_date_null_clears_value
  (create with a date, PATCH {"due_date": null}, assert 200 and due_date is None
   and due_state is None)
- test_patch_without_due_date_leaves_existing_date_unchanged
  (create with a date, PATCH {"title": "new title"}, assert due_date unchanged)

Overdue filter:
- test_overdue_filter_excludes_completed_late_and_future_tasks
  (create one overdue task, one completed_late task via the transition chain, one
   future-dated task, one no-date task; GET /tasks?overdue=true; assert exactly the
   overdue task is returned)
- test_overdue_filter_no_matches_returns_200_empty_list

Hard constraints:
- Use the existing TestClient fixtures and style. Do not create new fixtures unless
  truly needed.
- Do not mock storage or datetime. Use real past/future dates.
- To reach status Done, PATCH through valid transitions only (ToDo -> InProgress -> Done);
  never jump ToDo -> Done.
- Assert specific values, not just status codes, where the test name promises it.
- Do not modify app code, conftest.py, or existing tests.
- Output only the new test functions in one code block, ready to append to
  tests/test_tasks.py.

Test written by the agent:

```python
def test_create_task_with_valid_due_date_returns_201_and_echoes_date(client: TestClient):
    response = client.post(
        "/tasks",
        json={"title": "Task with due date", "due_date": "2026-07-30"},
    )
    assert response.status_code == 201
    body = response.json()
    assert body["due_date"] == "2026-07-30"
```

```
    assert body["due_state"] is None


def test_create_task_without_due_date_returns_201_with_null_due_date(client: TestClient):
    response = client.post("/tasks", json={"title": "Task without due date"})
    assert response.status_code == 201
    body = response.json()
    assert body["due_date"] is None
    assert body["due_state"] is None


def test_create_task_invalid_due_date_format_returns_422_naming_due_date(client:
TestClient):
    response = client.post(
        "/tasks",
        json={"title": "Bad due date", "due_date": "banana"},
    )
    assert response.status_code == 422
    body = response.json()
    assert isinstance(body["detail"], list)
    assert any(err["loc"][-1] == "due_date" for err in body["detail"])


def test_past_due_date_not_done_has_due_state_overdue(client: TestClient):
    response = client.post(
        "/tasks",
        json={"title": "Past due task", "due_date": "2020-01-01"},
    )
    assert response.status_code == 201
    body = response.json()
    assert body["due_date"] == "2020-01-01"
    assert body["status"] == "ToDo"
    assert body["due_state"] == "overdue"


def test_past_due_date_done_has_due_state_completed_late(client: TestClient):
    create_response = client.post(
        "/tasks",
        json={"title": "Completed late task", "due_date": "2020-01-01"},
    )
    assert create_response.status_code == 201
    task_id = create_response.json()["id"]

    patch_response = client.patch(f"/tasks/{task_id}", json={"status": "InProgress"})
```

```
    assert patch_response.status_code == 200
    assert patch_response.json()["status"] == "InProgress"

    final_response = client.patch(f"/tasks/{task_id}", json={"status": "Done"})
    assert final_response.status_code == 200
    body = final_response.json()
    assert body["status"] == "Done"
    assert body["due_date"] == "2020-01-01"
    assert body["due_state"] == "completed_late"


def test_patch_due_date_updates_value(client: TestClient):
    create_response = client.post("/tasks", json={"title": "Set due date later"})
    assert create_response.status_code == 201
    task_id = create_response.json()["id"]

    update_response = client.patch(
        f"/tasks/{task_id}",
        json={"due_date": "2026-08-01"},
    )
    assert update_response.status_code == 200
    body = update_response.json()
    assert body["due_date"] == "2026-08-01"
    assert body["due_state"] is None


def test_patch_due_date_null_clears_value(client: TestClient):
    create_response = client.post(
        "/tasks",
        json={"title": "Clear due date", "due_date": "2026-07-30"},
    )
    assert create_response.status_code == 201
    task_id = create_response.json()["id"]

    update_response = client.patch(
        f"/tasks/{task_id}",
        json={"due_date": None},
    )
    assert update_response.status_code == 200
    body = update_response.json()
    assert body["due_date"] is None
    assert body["due_state"] is None
```

```
def test_patch_without_due_date_leaves_existing_date_unchanged(client: TestClient):
    create_response = client.post(
        "/tasks",
        json={"title": "Keep due date", "due_date": "2026-07-30"},
    )
    assert create_response.status_code == 201
    task_id = create_response.json()["id"]

    update_response = client.patch(
        f"/tasks/{task_id}",
        json={"title": "new title"},
    )
    assert update_response.status_code == 200
    body = update_response.json()
    assert body["title"] == "new title"
    assert body["due_date"] == "2026-07-30"


def test_overdue_filter_excludes_completed_late_and_future_tasks(client: TestClient):
    overdue_task = client.post(
        "/tasks",
        json={"title": "Overdue only", "due_date": "2020-01-01"},
    )
    assert overdue_task.status_code == 201
    overdue_id = overdue_task.json()["id"]

    completed_late_task = client.post(
        "/tasks",
        json={"title": "Completed late task", "due_date": "2020-01-01"},
    )
    assert completed_late_task.status_code == 201
    completed_id = completed_late_task.json()["id"]

    client.patch(f"/tasks/{completed_id}", json={"status": "InProgress"})
    done_response = client.patch(f"/tasks/{completed_id}", json={"status": "Done"})
    assert done_response.status_code == 200
    assert done_response.json()["due_state"] == "completed_late"

    future_task = client.post(
        "/tasks",
        json={"title": "Future task", "due_date": "2026-08-15"},
    )
    assert future_task.status_code == 201
```

```
    no_date_task = client.post(
        "/tasks",
        json={"title": "No due date task"},
    )
    assert no_date_task.status_code == 201

    response = client.get("/tasks", params={"overdue": "true"})
    assert response.status_code == 200
    body = response.json()
    assert len(body) == 1
    assert body[0]["id"] == overdue_id
    assert body[0]["due_state"] == "overdue"


def test_overdue_filter_no_matches_returns_200_empty_list(client: TestClient):
    future_task = client.post(
        "/tasks",
        json={"title": "Future task only", "due_date": "2026-08-15"},
    )
    assert future_task.status_code == 201

    response = client.get("/tasks", params={"overdue": "true"})
    assert response.status_code == 200
    assert response.json() == []

========================================== 29 passed, 2 warnings in 0.39s ====
```

Break Tests:

Break Test 1 — the Done/completed_late rule:

1. In app/models.py, inside `due_state`, temporarily sabotage the Done check — e.g., change `TaskStatus.DONE` to `TaskStatus.TODO` in the comparison.
2. Run pytest. Expected failures: `test_past_due_date_done_has_due_state_completed_late` and `test_overdue_filter_excludes_completed_late_and_future_tasks` (the Done task now shows as "overdue" and leaks into the filter).
3. Record: what you broke, exact failing test names, one line of the failure output.
4. Restore, re-run, confirm 29 passed.

Break Test 2 — the due_date update path:

1. In app/storage.py's `update_task`, temporarily add `changes.pop("due_date", None)` right after the `model_dump` line.
2. Run pytest. Expected failures: `test_patch_due_date_updates_value` and `test_patch_due_date_null_clears_value`.
3. Record the same evidence, restore, re-run, confirm green.

FAILED tests/test_tasks.py::test_list_tasks_overdue_filter_returns_only_overdue - AssertionError: assert '3066140e365d...247c4bc243307' == '1eb4b1b6a73b...22833ee3fe139'
FAILED tests/test_tasks.py::test_past_due_date_not_done_has_due_state_overdue - AssertionError: assert 'completed_late' == 'overdue'
FAILED tests/test_tasks.py::test_past_due_date_done_has_due_state_completed_late - AssertionError: assert 'overdue' == 'completed_late'
FAILED tests/test_tasks.py::test_patch_due_date_updates_value - AssertionError: assert None == '2026-08-18'
FAILED tests/test_tasks.py::test_patch_due_date_null_clears_value - AssertionError: assert '2026-08-18' is None
FAILED tests/test_tasks.py::test_overdue_filter_excludes_completed_late_and_future_tasks - AssertionError: assert 'overdue' == 'completed_late'
========== 6 failed, 23 passed, 2 warnings in 0.63s ========

Now add the feature to the frontend prompt

I am working in frontend/index.html of my Task Tracker (branch: mid-course-project).

Current frontend (must be preserved):
- Vanilla HTML/CSS/JS Kanban board, three columns with data-status: ToDo, InProgress, Done.
- Priority sorting High -> Medium -> Low inside each column.
- Loading, empty, ready, and error states.
- Drag-and-drop that PATCHes /tasks/{id} with optimistic update and rollback on 422.
- Create/edit modal: POST /tasks and PATCH /tasks/{id}, title .trim() validation, server 422 shown in the modal, closes on Cancel/X/Escape/overlay click.

Backend already supports (do not change the backend):
- Optional due_date field: ISO date string "YYYY-MM-DD" or null.
- Every task response includes due_state: "overdue" | "completed_late" | null.
- GET /tasks?overdue=true returns only tasks whose due_state is "overdue".

STEP 1 ONLY — display layer. Do not add the filter yet.

Tasks:

1. Modal: add an optional due date input (type="date") between the existing fields.
   - Create mode: empty by default; if left empty, send "due_date": null.
   - Edit mode: pre-fill from the task's due_date; clearing the input sends "due_date": null on submit.
   - A server 422 on due_date must show in the modal without closing it, same as existing validation errors.
2. Cards: if a task has a due_date, show it as YYYY-MM-DD text on the card; if null, render no date element at all.
3. Pills: if due_state is "overdue", show a pill element with the exact text "Overdue"; if "completed_late", a pill with the exact text "Completed late"; if null, no pill. Style the two pills distinctly (e.g., red-ish for Overdue, neutral/grey for Completed late) reusing existing CSS patterns.

Constraints:
- Do not break drag-and-drop, sorting, UI states, or existing modal behavior.
- Do not add frameworks, libraries, or new files.
- Escape all task-derived text before inserting into the DOM, consistent with existing rendering.
- Return a focused diff: only the modal markup, renderCard changes, submit handler changes, and new CSS.

Feature 2:

Prompt strong:

You are a senior backend developer helping me evaluate lightweight architectures for a learning project. Context: I am building a Task Tracker application with a Python/FastAPI backend and a simple web frontend and now adding the feature of Task comments Reviewed requirements:

Story 1: As a team member, I want to add a comment to a task so that I can record notes or updates about the work. Acceptance Criteria:

- `POST /tasks/{task_id}/comments` with a non-blank `text` field creates a comment and returns HTTP 201 with the comment body including `id`, `task_id`, `text`, and `created_at`.
- A blank or whitespace-only `text` returns HTTP 422 with a detail message that names the `text` field.
- Posting a comment to a missing `task_id` returns HTTP 404 with a detail message identifying the task.

Story 2: As a team member, I want to view all comments on a task so that I can follow the history of updates. Acceptance Criteria:

- `GET /tasks/{task_id}/comments` returns HTTP 200 with a list of comments for that task, ordered by `created_at` ascending.
- A test that creates two comments in sequence asserts the first created comment appears at index 0 in the response list.
- If the task exists but has no comments, the response is HTTP 200 with an empty list, not 404.
- Requesting comments for a missing `task_id` returns HTTP 404.

Story 3: As a team member, I want to delete a comment so that I can remove notes that are no longer relevant. Acceptance Criteria:

- `DELETE /tasks/{task_id}/comments/{comment_id}` returns HTTP 204 with no response body on success.
- If the `task_id` does not exist, returns HTTP 404 naming the task.
- If the `comment_id` does not exist on that task, returns HTTP 404 naming the comment.
- If `comment_id` exists but belongs to a different `task_id`, returns HTTP 404 naming the comment.

Story 5: As a team member, I want to read and add comments inside the task edit modal so that I can manage notes without leaving the board. Acceptance Criteria:

- Opening the edit modal for a task loads and displays its existing comments, each showing `text` and `created_at`.
- Submitting a blank or whitespace-only comment is blocked client-side before any network request, with a visible message.
- A server 422 on `text` is shown inside the modal without closing it.
- After a comment is successfully added, the comment list in the modal refreshes.
- Delete is available per comment; after the delete request returns 204 the comment disappears from the list immediately.

Constraints:

- This is a learning project, not production software.
- The backend must use Python, FastAPI, and Pydantic for validation.
- I want a REST API backend and a separate simple web frontend.
- Keep the tech stack simple, well-documented, and easy to run locally.
- No authentication or multi-tenancy.
- Do not suggest microservices, Docker, cloud deployment, or production database setup.

Task: Propose two different lightweight architectures:

- Option A should be the simplest local-storage approach appropriate for a first learning project.
- Option B may use a lightweight local database approach if it improves realism without overcomplicating the project.

For each option, provide:

1. Tech stack and data storage choice
2. Folder structure
3. Data model sketch with Pydantic fields and constraints
4. Three trade-offs compared to the other option

Output format: Return Option A and Option B in clearly separated sections. Do not choose for me.

Response to strong prompt:

Same framing note as Feature 1: you're not starting fresh — you have a working codebase. So both options describe how to add comments within what exists, not a full rewrite.

Option A — Separate in-memory dict for comments

1. Tech stack and data storage

- Python, FastAPI, Pydantic v2 — unchanged.
- New module-level dict in a new file `app/comments_storage.py`: `_comments: dict[str, CommentResponse] = {}`
- Comments keyed by `comment_id`; `task_id` stored as a field on each comment so filtering and ownership checks work without nesting.

2. Folder structure (delta only)

```
None
app/
  main.py               # add 3 new routes: POST, GET, DELETE /tasks/{id}/comments
  models.py             # add CommentCreate, CommentResponse
  storage.py            # unchanged
  comments_storage.py   # NEW: _comments dict + 4 helper functions
  business_rules.py     # unchanged
```

```
tests/
  test_tasks.py         # add comment tests
  test_comments.py      # optional: separate file for comment tests
docs/midcourse/         # documentation
```

3. Data model sketch

Python

```python
# models.py additions (Pydantic v2)

class CommentCreate(BaseModel):
    model_config = ConfigDict(extra="forbid")
    text: str

    @field_validator("text")
    @classmethod
    def text_must_not_be_blank(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("text must not be blank")
        return v.strip()

class CommentResponse(BaseModel):
    model_config = ConfigDict(extra="forbid")
    id: str
    task_id: str
    text: str
    created_at: datetime
```

Python

```python
# comments_storage.py

_comments: dict[str, CommentResponse] = {}

def add_comment(task_id: str, payload: CommentCreate) ->
CommentResponse
```

```
def get_comments_by_task(task_id: str) -> list[CommentResponse]
    # returns list sorted by created_at ascending
def delete_comment(task_id: str, comment_id: str) -> bool
    # returns False if comment_id missing OR belongs to different
task_id
def _reset_comments() -> None
    # clears _comments for tests only
```

4. Three trade-offs vs. Option B

- Simpler: zero new dependencies, no schema design, tests keep using `_reset_comments()` exactly like `_reset()`. The existing conftest pattern extends directly.
- Less realistic: comments wipe on restart like tasks do; no referential integrity between tasks and comments at the storage level — if a task is deleted, its comments stay orphaned in the dict unless the delete route explicitly cleans them up.
- Lower risk for the deadline: the diff is isolated to two new files and three new routes. Nothing existing needs rewriting.

Option B — SQLite with two tables

1. Tech stack and data storage

- Python, FastAPI, Pydantic v2, plus Python's built-in `sqlite3` module.
- Two tables in `tasks.db`: the existing `tasks` table (rewritten from in-memory) and a new `comments` table with a foreign-key-style `task_id` column.
- No ORM — raw SQL inside storage functions.

2. Folder structure

```
None
app/
  main.py               # same 3 new comment routes
  models.py             # same CommentCreate, CommentResponse
additions
  db.py                 # NEW: connection helper, CREATE TABLE
statements
  storage.py            # REWRITTEN: tasks stored in SQL
```

```
  comments_storage.py   # NEW: comments stored in SQL
  business_rules.py     # unchanged
tasks.db                # local file — add to .gitignore
tests/
  conftest.py           # CHANGED: _reset now truncates tables or
uses temp db
  test_tasks.py         # all existing tests must be re-verified
  test_comments.py      # new comment tests
```

3. Data model sketch

Pydantic models are identical to Option A. The difference is beneath them:

```SQL
CREATE TABLE IF NOT EXISTS comments (
  id TEXT PRIMARY KEY,
  task_id TEXT NOT NULL,
  text TEXT NOT NULL,
  created_at TEXT NOT NULL
);
```

Storage functions keep the same signatures as Option A — routes and tests don't change — but internally use SQL instead of dict operations.

4. Three trade-offs vs. Option A

- More realistic: comments survive restarts, and the `task_id` relationship is explicit in the schema. Deleting a task could cascade to comments with a SQL `ON DELETE CASCADE` rule.
- Much more work and risk: Option B requires rewriting the entire `storage.py` layer to SQL, which means re-verifying all 29 existing tests against the new storage — four days before the deadline. The comment feature alone doesn't justify this.
- Test setup gets heavier: `_reset_comments()` becomes table truncation or per-test temp databases, and date-string vs. datetime comparisons in SQL are a new failure class your current tests don't encounter.

Questions you should decide, not me:

1. Does SQLite's persistence benefit outweigh re-verifying the full existing test suite this week?
2. The orphaned-comment problem in Option A (task deleted, comments remain) — is that worth handling explicitly in the DELETE /tasks/{id} route, or acceptable to leave for a learning project?
3. Same question as Feature 1: the project grades workflow quality over feature ambition — does Option B's realism earn anything the rubric measures?

Write the ADR yourself and send it for a checklist review when ready.

Reject story 4 **No edit comment.** It assumed comments are add-only and delete-only — no `PATCH /comments/{id}`. The brief doesn't mention editing; adding it would be scope creep.

Story 1: As a team member, I want to add a comment to a task so that I can record notes or updates about the work.

Acceptance Criteria:

- `POST /tasks/{task_id}/comments` with a non-blank `text` field creates a comment and returns HTTP 201 with the comment body including `id`, `task_id`, `text`, and `created_at`.
- A blank or whitespace-only `text` returns HTTP 422 with a detail message that names the `text` field.
- Posting a comment to a missing `task_id` returns HTTP 404 with a detail message identifying the task.

Story 2: As a team member, I want to view all comments on a task so that I can follow the history of updates.

Acceptance Criteria:

- `GET /tasks/{task_id}/comments` returns HTTP 200 with a list of comments for that task, ordered by `created_at` ascending.
- If the task exists but has no comments, the response is HTTP 200 with an empty list, not 404.
- Requesting comments for a missing `task_id` returns HTTP 404.

Story 3: As a team member, I want to delete a comment so that I can remove notes that are no longer relevant.

Acceptance Criteria:

- `DELETE /tasks/{task_id}/comments/{comment_id}` returns HTTP 204 with no response body on success.
- If the `task_id` does not exist, returns HTTP 404 naming the task.
- If the `comment_id` does not exist on that task, returns HTTP 404 naming the comment.

Story 4: As a team member, I want to see the comment count on each task card so that I can tell at a glance which tasks have discussion without opening them.

Acceptance Criteria:

- Cards with one or more comments display the count as a number (e.g., 💬 `3`).
- Cards with zero comments show no comment indicator, or display 💬 `0` — consistent across all cards.
- The count updates after a comment is added or deleted without requiring a full page reload.

Story 5: As a team member, I want to read and add comments inside the task edit modal so that I can manage notes without leaving the board.

Acceptance Criteria:

- Opening the edit modal for a task loads and displays its existing comments, each showing `text` and `created_at`.
- Submitting a blank or whitespace-only comment is blocked client-side before any network request, with a visible message.
- A server 422 on `text` is shown inside the modal without closing it.
- After a comment is successfully added or deleted, the comment list in the modal refreshes and the card count updates.
- Delete is available per comment; after confirming deletion the comment disappears from the list immediately.

Strong prompt for User stories table

You are a senior developer reviewing requirements before implementation.

Context: The requirements should support a simple FastAPI backend and simple web frontend. The app has no authentication, no user accounts, no multi-tenancy, no real-time updates, no mobile app, and no notifications.

Task: Review the user stories below for testability and scope alignment. Check each story for:

- Is the story in scope?
- Are the acceptance criteria specific and testable?
- Is there at least one meaningful edge case or failure case across the set?
- Are any words too vague, such as "works", "properly", or "as expected"?
- Did the AI assume a feature that was not requested?

Constraints:

- Do not generate a new story set.
- Suggest minimal edits only.

Response table:

Story IDPass / Needs revisionIssueMinimal editStory 1PassIn scope, all three criteria are specific and testable. The 422 names the field, the 404 identifies the task. Failure cases covered.None required.Story 2Needs revision (minor)"ordered by `created_at` ascending" is testable but only if the test creates at least two comments with a known sequence. Worth stating the test expectation explicitly. Also no failure case for an invalid `task_id` format — but FastAPI returns 422 for that automatically, so acceptable to leave implicit.Add one criterion: "A test that creates two comments in sequence and asserts the first appears at index 0 in the response list."Story 3Needs revisionMissing edge case: what happens if `comment_id` exists in the system but belongs to a *different* task than `task_id`? The current criteria don't say. This is a real implementation decision — the route could return 404 or 204.Add: "If `comment_id` exists but belongs to a different `task_id`, returns HTTP 404 naming the comment."Story 5Needs revision (two issues)(1) "After confirming deletion" implies a JS confirmation dialog (e.g., `window.confirm()`), which was not requested and is not testable in pytest. (2) "the card count updates" references the dropped Story 4 — must be removed.Remove "after confirming deletion" — replace with "after the delete request returns 204". Remove "and the card count updates" entirely.

Result to editing the user stories

Story 1: As a team member, I want to add a comment to a task so that I can record notes or updates about the work.

Acceptance Criteria:

- `POST /tasks/{task_id}/comments` with a non-blank `text` field creates a comment and returns HTTP 201 with the comment body including `id`, `task_id`, `text`, and `created_at`.
- A blank or whitespace-only `text` returns HTTP 422 with a detail message that names the `text` field.
- Posting a comment to a missing `task_id` returns HTTP 404 with a detail message identifying the task.

Story 2: As a team member, I want to view all comments on a task so that I can follow the history of updates.

Acceptance Criteria:

- `GET /tasks/{task_id}/comments` returns HTTP 200 with a list of comments for that task, ordered by `created_at` ascending.
- A test that creates two comments in sequence asserts the first created comment appears at index 0 in the response list.
- If the task exists but has no comments, the response is HTTP 200 with an empty list, not 404.
- Requesting comments for a missing `task_id` returns HTTP 404.

Story 3: As a team member, I want to delete a comment so that I can remove notes that are no longer relevant.

Acceptance Criteria:

- `DELETE /tasks/{task_id}/comments/{comment_id}` returns HTTP 204 with no response body on success.
- If the `task_id` does not exist, returns HTTP 404 naming the task.
- If the `comment_id` does not exist on that task, returns HTTP 404 naming the comment.
- If `comment_id` exists but belongs to a different `task_id`, returns HTTP 404 naming the comment.

Story 5: As a team member, I want to read and add comments inside the task edit modal so that I can manage notes without leaving the board.

Acceptance Criteria:

- Opening the edit modal for a task loads and displays its existing comments, each showing `text` and `created_at`.
- Submitting a blank or whitespace-only comment is blocked client-side before any network request, with a visible message.
- A server 422 on `text` is shown inside the modal without closing it.
- After a comment is successfully added, the comment list in the modal refreshes.
- Delete is available per comment; after the delete request returns 204 the comment disappears from the list immediately.

ADR

ADR - Option A written by me

We are adding a new feature to our task management API that will enable users to add comments to tasks. We must choose how to handle and store this new data. A basic in-memory storage method is used in our current application. Due to an impending deadline, we have to decide between moving to a persistent SQLite database (Option B) or continuing with this in-memory method (Option A).

We proceed with Option A.

We decided to use a separate in-memory dictionary for comments, keying them by `comment_id` and storing the `task_id` directly on the comment object for easy filtering. We chose this because it is the simplest option, it keeps our existing pytest suite and reset fixture pattern working seamlessly (just adding a `_reset_comments()` helper), the app still runs locally without complex setup, and the diff is tightly isolated to a few new files and routes without touching the existing task storage.
During planning we rejected or corrected two AI suggestions.

First, the AI proposed a SQLite-based storage layer with two tables (Option B); we rejected it because rewriting the entire storage foundation and re-verifying all 29 existing tests four days before the deadline added massive risk that the feature alone didn't justify. Second, the AI initially left the "orphaned-comment problem" unresolved, where comments would remain stuck in the dictionary if their parent task was deleted; we corrected this so that deleting a task explicitly cascades to delete all associated comments, manually maintaining referential integrity in the route.

Agent prompts to code feature 1:

I am working in my Task Tracker repository (branch: mid-course-project) for an AI-Assisted Coding course.

Current project state:

- Backend: Python/FastAPI, Pydantic v2, in-memory storage in app/storage.py (_tasks dict).
- Files: app/main.py (5 CRUD routes + due_date feature), app/models.py,
  app/storage.py, app/business_rules.py, tests/test_tasks.py (29 passing tests).
- Feature 1 (due dates) is already complete and tested. DO NOT touch it.
- Frontend: frontend/index.html, vanilla Kanban board with drag-and-drop,
  modal, due date display, and overdue filter toggle.

Feature I am adding now (decided, do not propose alternatives):
Task Comments — add/list/delete comments on tasks.

Architecture decisions already made:
- Storage: a NEW separate file app/comments_storage.py with its own in-memory
  dict: _comments: dict[str, CommentResponse] = {}
- Comments keyed by comment_id; task_id stored as a field on each comment.
- DO NOT embed comments inside TaskResponse or modify app/storage.py.
- DO NOT add SQLite, a database, ORM, or any new pip dependency.
- Deleting a task (DELETE /tasks/{id}) must cascade-delete all comments for
  that task — update that route only for this cascade, nothing else.

New models to add to app/models.py:
- CommentCreate:
    model_config = ConfigDict(extra="forbid")
    text: str — required, strip whitespace, reject blank with 422 naming text field
- CommentResponse:
    model_config = ConfigDict(extra="forbid")
    id: str
    task_id: str
    text: str
    created_at: datetime

New file app/comments_storage.py — define exactly these functions:
- add_comment(task_id: str, payload: CommentCreate) -> CommentResponse
- get_comments_by_task(task_id: str) -> list[CommentResponse]
    sorted by created_at ascending
- delete_comment(task_id: str, comment_id: str) -> bool
    returns False if comment_id missing OR belongs to a different task_id
- _reset_comments() -> None
    clears _comments for tests only

New routes to add to app/main.py:
- POST   /tasks/{task_id}/comments → 201, CommentResponse, tags=["comments"]
    check task exists first (404 if not), then add_comment
- GET   /tasks/{task_id}/comments → 200, list[CommentResponse], tags=["comments"]
    check task exists first (404 if not), then get_comments_by_task

- DELETE /tasks/{task_id}/comments/{comment_id} → 204, tags=["comments"]
  check task exists (404), then delete_comment;
  if False returned raise 404 naming the comment

Workflow rules:
- One file per step. Start with models.py, then comments_storage.py,
  then the three routes in main.py, then the cascade in DELETE /tasks/{id}.
- Do not rewrite whole files.
- Do not touch storage.py, business_rules.py, or any existing route
  except adding the cascade to DELETE /tasks/{id}.
- Treat output as a draft; I will inspect, run, and test before accepting.

Test written by the agent:

```
from datetime import datetime, timezone, timedelta, date

from fastapi.testclient import TestClient

from app.comments_storage import get_comments_by_task


def test_create_comment_for_existing_task_returns_201(client: TestClient):
    task_response = client.post("/tasks", json={"title": "Task with
comment"})
    assert task_response.status_code == 201
    task_id = task_response.json()["id"]

    response = client.post(
        f"/tasks/{task_id}/comments",
        json={"text": "This is a comment."},
    )

    assert response.status_code == 201
    body = response.json()
    assert body["task_id"] == task_id
    assert body["text"] == "This is a comment."
    assert body["id"]
    assert body["created_at"]


def test_create_comment_blank_text_returns_422_naming_text(client:
TestClient):
```

```
    task_response = client.post("/tasks", json={"title": "Task with
invalid comment"})
    assert task_response.status_code == 201
    task_id = task_response.json()["id"]

    response = client.post(
        f"/tasks/{task_id}/comments",
        json={"text": "   "},
    )

    assert response.status_code == 422
    body = response.json()
    assert isinstance(body["detail"], list)
    assert any(err["loc"][-1] == "text" for err in body["detail"])


def test_create_comment_for_missing_task_returns_404(client: TestClient):
    response = client.post(
        "/tasks/nonexistent/comments",
        json={"text": "Hello"},
    )
    assert response.status_code == 404
    assert response.json()["detail"] == "Task with id nonexistent not
found"


def
test_list_comments_returns_200_and_empty_list_for_task_with_no_comments(cl
ient: TestClient):
    task_response = client.post("/tasks", json={"title": "Task with no
comments"})
    assert task_response.status_code == 201
    task_id = task_response.json()["id"]

    response = client.get(f"/tasks/{task_id}/comments")
    assert response.status_code == 200
    assert response.json() == []
```

```
def test_list_comments_returns_comments_sorted_by_creation_time(client:
TestClient):
    task_response = client.post("/tasks", json={"title": "Task with
multiple comments"})
    assert task_response.status_code == 201
    task_id = task_response.json()["id"]

    first = client.post(
        f"/tasks/{task_id}/comments",
        json={"text": "First comment"},
    )
    second = client.post(
        f"/tasks/{task_id}/comments",
        json={"text": "Second comment"},
    )

    assert first.status_code == 201
    assert second.status_code == 201

    response = client.get(f"/tasks/{task_id}/comments")
    assert response.status_code == 200
    body = response.json()
    assert len(body) == 2
    assert body[0]["id"] == first.json()["id"]
    assert body[1]["id"] == second.json()["id"]


def test_delete_comment_returns_204_and_comment_is_removed(client:
TestClient):
    task_response = client.post("/tasks", json={"title": "Task with
deleteable comment"})
    assert task_response.status_code == 201
    task_id = task_response.json()["id"]

    comment_response = client.post(
        f"/tasks/{task_id}/comments",
        json={"text": "Comment to delete"},
    )
    assert comment_response.status_code == 201
    comment_id = comment_response.json()["id"]
```

```
    response = client.delete(f"/tasks/{task_id}/comments/{comment_id}")
    assert response.status_code == 204
    assert response.content == b""

    list_response = client.get(f"/tasks/{task_id}/comments")
    assert list_response.status_code == 200
    assert list_response.json() == []


def test_delete_comment_with_wrong_task_returns_404(client: TestClient):
    first_task = client.post("/tasks", json={"title": "First task"})
    second_task = client.post("/tasks", json={"title": "Second task"})
    assert first_task.status_code == 201
    assert second_task.status_code == 201
    first_id = first_task.json()["id"]
    second_id = second_task.json()["id"]

    comment_response = client.post(
        f"/tasks/{first_id}/comments",
        json={"text": "First task comment"},
    )
    assert comment_response.status_code == 201
    comment_id = comment_response.json()["id"]

    wrong_delete =
client.delete(f"/tasks/{second_id}/comments/{comment_id}")
    assert wrong_delete.status_code == 404
    assert wrong_delete.json()["detail"] == f"Comment with id {comment_id}
not found"


def test_delete_task_cascades_comments(client: TestClient):
    task_response = client.post("/tasks", json={"title": "Task to delete
with comments"})
    assert task_response.status_code == 201
    task_id = task_response.json()["id"]

    comment_response = client.post(
        f"/tasks/{task_id}/comments",
```

```
        json={"text": "Will be deleted with task"},
    )
    assert comment_response.status_code == 201

    delete_response = client.delete(f"/tasks/{task_id}")
    assert delete_response.status_code == 204

    assert get_comments_by_task(task_id) == []
```

Break Tests:

Did not find theses test should be added:
`GET /tasks/{missing_id}/comments` → 404
`DELETE /tasks/{missing_id}/comments/{comment_id}` → 404 (task missing)
`DELETE /tasks/{valid_id}/comments/{nonexistent_comment_id}` → 404 (comment missing)

= 11 passed in 0.32s ==

```
>       assert wrong_delete.status_code == 404
E       assert 204 == 404
E        +  where 204 = <Response [204 No Content]>.status_code

test_comments.py:129: AssertionError
=========================================================== short test summary info ============================
FAILED test_comments.py::test_delete_comment_with_wrong_task_returns_404 - assert 204 == 404
============================================================ 1 failed, 10 passed in 0.55s =====================
(venv) PS C:\Users\hp\Desktop\BLR\AAC\task-tracker\task-tracker-api\tests>
```

test_comments.py 1, U comments_storage.py U

app > comments_storage.py > delete_comment

```
27
28  def delete_comment(task_id: str, comment_id: str) -> b
29      comment = _comments.get(comment_id)
30      #if comment is None or comment.task_id != task_id:
31      #    return False
32      if comment is None:
33          return False
34      _comments.pop(comment_id)
```

Now add the feature to the frontend prompt

I am working in frontend/index.html of my Task Tracker (branch: mid-course-project).

Current frontend (must be preserved):
- Vanilla HTML/CSS/JS Kanban board, three columns: ToDo, InProgress, Done.
- Priority sorting High -> Medium -> Low inside each column.
- Loading, empty, ready, and error states.
- Drag-and-drop that PATCHes /tasks/{id} with optimistic update and rollback on 422.
- Create/edit modal: POST /tasks and PATCH /tasks/{id}, title .trim() validation, due date field, server 422 shown without closing modal, closes on Cancel/X/Escape/overlay click.
- Overdue filter toggle in the header.

Backend comment routes already working (do not change the backend):
- POST /tasks/{task_id}/comments → 201, body: {id, task_id, text, created_at}
- GET /tasks/{task_id}/comments → 200, list sorted by created_at ascending
- DELETE /tasks/{task_id}/comments/{comment_id} → 204, no body

TASK: Add a comments section inside the edit modal ONLY.
Do NOT add comments to create mode. Do NOT add comment count to cards.

Comments section behavior:
1. When the edit modal opens for an existing task:
   - Immediately fetch GET /tasks/{task_id}/comments and render the list below the existing form fields, before the action buttons.
   - Each comment shows: text and created_at formatted as YYYY-MM-DD HH:MM.
   - Each comment has a Delete button.
   - If no comments exist, show a short placeholder text e.g. "No comments yet."
   - If the fetch fails, show a brief error message inside the modal; do not close the modal.

2. Add new comment:
   - Below the comment list, add a single-line text input and an "Add Comment" button.
   - Client-side: if input is empty or whitespace-only after .trim(), show "Comment cannot be blank" and do not send any request.
   - On submit: POST /tasks/{task_id}/comments with {"text": trimmedValue}.
   - On 201: clear the input, refresh the comment list by re-fetching GET /tasks/{task_id}/comments.
   - On 422: show the server detail message near the input without closing the modal.

3. Delete a comment:
   - Each comment's Delete button sends DELETE /tasks/{task_id}/comments/{comment_id}.
   - On 204: remove that comment from the list immediately without re-fetching.
   - On failure: show a brief error message near that comment; do not close modal.

Constraints:
- Do not break drag-and-drop, sorting, UI states, or any existing modal behavior.
- Do not add comments section to create mode (taskId is null in create mode — skip the comments section entirely).
- Do not add frameworks, libraries, or new files.
- Do not show comment count on cards.
- Escape all comment text before inserting into the DOM.
- Keep the diff focused: modal open handler, comment fetch/render function, add-comment submit handler, delete-comment handler, and minimal new CSS.
- Do not rewrite the whole file.
- Return a focused diff.

Frontend problems:

1-there is a problem when i add a comment i ca not scroll up or click okay or go back to the dashboard
2-the comments now are not appearing on the main dashboard